Tensor Omics (TOX)

Analyze gene expression data and evolutionary relationships between genes

User's Guide

Vivian Bass[1], **Anna Beketova**[1], **Luka Faensen**[1], **Asis Hallab**[1], **Aaron Schröder**[1], **Franz-Eric Sill**[1]

[1]University of Applied Sciences Bingen, Germany

Last revised August 25, 2025

Contents

1 Tensor Omics Method and Theory

The Tensor Omics (TOX) tool implements a streamlined pipeline for analyzing gene expression data and evolutionary relationships between genes. The pipeline integrates tools for data normalization, and geometric analysis of gene expression vectors. This document outlines the current implementation details and provides the mathematical formulation of the algorithms used in the prototype.

1.1 The TOX Data Table

The foundational data structure in Tensor Omics is the **Data Table**. Unlike a simple spreadsheet, a Data Table is a conceptual grouping of multiple, type-specific 2D arrays that represent datasets read from a source like a CSV file. Because Fortran is a strongly-typed language, data of different types (e.g., integers, reals, characters) cannot be stored in a single matrix. The TOX library solves this by parsing the source data into a collection of distinct, column-major arrays that share the same number of rows.

The process begins by reading the entire CSV file into a single, general-purpose 2D array of strings. This string array acts as an in-memory staging area. From this staging array, users can then extract specific columns into strongly-typed arrays, creating the components of the Data Table:

- `integer_data(:,:)`: An array for integer columns (e.g., gene IDs).
- `real_data(:,:)`: An array for real-valued columns (e.g., expression levels, scores).
- `logical_data(:,:)`: An array for boolean columns (e.g., status flags).
- `character_data(:,:)`: An array for string columns (e.g., gene names).
- `complex_data(:,:)`: An array for complex number columns.

All downstream analyses in TOX operate on these strongly-typed arrays.

1.2 From Raw Data to Expression Vectors

The starting point for all downstream analyses is a table of gene-wise expression measurements across a set of biological samples, such as tissues, developmental stages, or experimental conditions. These measurements may originate from transcriptomic, proteomic, or metabolomic experiments, typically normalized to account for sequencing depth and technical variation.

What is Gene Expression Data? For each gene, a real-valued quantity is recorded that reflects its biological activity in a given sample. In the case of transcriptomics, this is often the abundance of messenger RNA (mRNA) molecules transcribed from that gene, estimated via sequencing and mapped back to the gene's known sequence. The result is a non-negative real number proportional to the gene's transcriptional activity in that sample. After normalization and transformation, these values can be treated as coordinates in a real vector space.

Thus, each gene is represented as a vector $\vec{v} \in \mathbb{R}^n$, where n is the number of biological samples (e.g., tissues or time points), and each component $v_i \geq 0$ corresponds to the expression level in sample i. These vectors encode the distribution of gene activity across conditions and form the geometric basis for all downstream analyses.

Note on Stress Responses. In certain studies, a subset of the samples may correspond to stress conditions. Here, "stress" refers to any external or internal perturbation—such as mechanical damage, heat shock, pathogen infection, or nutrient deprivation—that triggers measurable changes in gene expression. These are often analyzed relative to baseline conditions using log fold-change transformations, such that the resulting vectors reflect differential expression patterns, including up- and downregulation across dimensions.

1.3 Normalization of Expression Values

Raw gene expression data is normalized in the following steps:

1. Division by gene-wise standard deviation for scale normalization.
2. Quantile normalization across samples to reduce global expression biases.
3. Log-transformation: $x \mapsto \log(1 + x)$ to stabilize variance and suppress outliers.
4. For stress response studies, log fold-changes are computed:

$$\Delta_{\text{stress}} = \log(1 + x_{\text{stress}}) - \log(1 + x_{\text{control}})$$

   This effectively makes each stress related axis show the fold change in gene expression under stress, e.g. "drought in root divided by control root".

1.4 Calculation of family Centroids

To summarize the expression profile of each family, Tensor Omics computes a centroid vector for every group. The centroid represents the average expression pattern of all genes assigned to the family across all conditions or tissues.

There are two modes for selecting the set of genes to include in the centroid:

- **Full mode:** All genes assigned to the family are used to compute the centroid (mean vector).
- **Orthologs-only mode:** Only genes explicitly marked as orthologs are included in the centroid calculation. This mode is useful for focusing on conserved gene relationships across species.

1.5 Euclidean Distance Between Expression Vectors

The Euclidean distance is used throughout Tensor Omics to quantify the geometric difference between two gene expression vectors, or between a gene and its family centroid. Given two vectors $\vec{v}_1, \vec{v}_2 \in \mathbb{R}^n$, the Euclidean distance is defined as:

$$d(\vec{v}_1, \vec{v}_2) = \sqrt{\sum_{i=1}^{n} (v_{1,i} - v_{2,i})^2}$$

This metric measures the straight-line distance between two points in n-dimensional space. In the context of gene expression, it reflects the overall magnitude of difference in expression profiles across all samples or conditions.
For example, the distance between a paralog's expression vector and the centroid of its gene family is used to detect outlier genes and to quantify the degree of divergence.

1.6 Outlier Detection Based on Family-Scaled Distances

To robustly identify genes with divergent expression profiles within their families, Tensor Omics implements an outlier detection procedure based on family-scaled Euclidean distances. The process consists of the following steps:

1. **Compute Euclidean Distances:** For each gene, calculate the Euclidean distance d_i between its expression vector and the centroid of its assigned family (see Section 1.5).

2. **Estimate Family-Specific Scaling Factors:** For each family f, we compute a family-specific scaling factor. This can be the maximum distance between any ortholog and the ortholog centroid. For families lacking sufficient orthologs, a LOESS smoothing strategy is used to estimate an appropriate scaling factor based on the family's median intra-member distance.

3. **Relative Distance Index (RDI):** For each gene i in family f, compute the Relative Distance Index (RDI):
$$\text{RDI}_i = \frac{d_i}{d_{scale}}$$
This normalizes the gene's distance by the expected intra-family variability, making the metric comparable across families of different sizes and variances.

4. **Outlier Detection:** Genes are flagged as outliers if their RDI exceeds a chosen percentile threshold (typically the 95th percentile) of the empirical RDI distribution. That is, gene i is an outlier if $\text{RDI}_i \geq T$, where T is the threshold value corresponding to the desired percentile.

This approach ensures that outlier detection is adaptive to the structure and variability of each gene family, and is robust to differences in family size and dispersion. The LOESS-based scaling is particularly important for families with few members.

1.7 Tissue Versatility and Specificity

Tissue Versatility quantifies how uniformly a gene is expressed across all tissues (or, more generally, axes of the expression space). It is defined as the cosine of the angle between a gene's expression vector $\vec{v}$ and the space diagonal $\vec{d} = (1, 1, \ldots, 1)$. The smaller this angle (the larger the cosine), the greater the tissue (axis) versatility of the gene's expression.

Interpretation:

- A lower angle (cosine closer to 1) means the gene is more evenly expressed across all tissues (high versatility).
- A larger angle (cosine closer to 0) means the gene is active in only a few tissues (high specificity).

Dimensionality Dependence

The maximum possible angle to the diagonal depends on the number of tissues n. The cosine of this angle is minimal when a gene is expressed in only one tissue:

$$\cos(\varphi_{\text{max\_angle}}) = \frac{1}{\sqrt{n}}$$

Normalized Tissue Versatility

To make versatility comparable across datasets with different numbers of tissues, we define a normalized version that scales values into the interval $[0, 1]$:

$$\text{Tissue Versatility} = \frac{\cos(\varphi) - \frac{1}{\sqrt{n}}}{1 - \frac{1}{\sqrt{n}}}$$

This maps the cosine values into the interval $[0, 1]$:

- 1: perfectly uniform expression (maximum versatility)
- 0: expression in only one tissue (minimum versatility)

Tissue Specificity

Tissue specificity is defined as the complement of normalized tissue versatility:

$$\text{Tissue Specificity} = 1 - \text{Tissue Versatility}$$

Values close to 1 indicate high tissue-specificity; values close to 0 indicate broad, uniform expression.

2 Error Handling

In case you encounter errors while using TOX, the following error codes may help you identify the issue. The error codes are grouped into categories based on their nature, such as I/O errors, format validation errors, memory errors, and internal errors.

Table 1: TOX Error Codes Reference

Error Code	Description
0	Success
1xx: I/O / File Reading Errors	
101	Could not open file.
102	Could not read magic number.
103	Could not read array type code.
104	Could not read number of dimensions.
105	Could not read array dimensions.
106	Could not read character length.
107	Could not read array data.
2xx: Format / Input Validation Errors	
200	Invalid format detected (e.g., magic mismatch or corrupt format).
201	Invalid input provided.
202	Empty input arrays provided (e.g., zero-length arrays).
203	Dimension mismatch detected (shape inconsistencies).
204	NaN or Inf found in input data where not allowed.
205	Unsupported data type encountered (e.g., complex types).

Continued on next page

Table 1 – continued from previous page

Error Code	Description
3xx: Memory Errors	
301	Memory allocation failed.
302	Null pointer reference encountered.
5xxx: Fortran Runtime / Unit State Errors	
5002	Fortran runtime error: unit not open or not connected.
9xxx: Internal / Unknown Errors	
9001	Internal error: unexpected state or invariant violated.
9999	Unknown error (fallback for unexpected errors).

3 Implementation Notes

This section details the practical pipeline for using the TOX library to read and process tabular data. The workflow is consistent across Fortran, Python, and R, starting with reading a CSV file into a staging area of strings and then converting columns to their proper data types.

3.1 Fortran Pipeline

In a pure Fortran application, the user follows a two-step process to construct the Data Table.

1. **Read CSV to Strings:** First, call the `read_csv_to_strings` subroutine. This reads the entire CSV file into a 2D allocatable array of characters, which serves as the in-memory source for all subsequent operations.

2. **Extract Typed Columns:** Next, call the specific `read_*_columns` subroutines (e.g., `read_integer_columns` `read_real_columns`) to parse columns from the string array into new, strongly-typed 2D arrays.

Listing 1: Fortran Data Loading Workflow

```
PROGRAM tox_fortran_example
    USE csv_file_reader_module, ONLY: read_csv_to_strings
    USE csv_read_int_module, ONLY: read_integer_columns
    USE csv_read_real_module, ONLY: read_real_columns
    USE csv_parser_module, ONLY: MAX_FIELD_LEN
    IMPLICIT NONE

    CHARACTER(LEN=MAX_FIELD_LEN), ALLOCATABLE :: data_str(:,:), header_str
        (:)
    INTEGER, ALLOCATABLE :: data_int(:,:)
    REAL(8), ALLOCATABLE :: data_real(:,:)
    INTEGER :: status

    CHARACTER(LEN=*), PARAMETER :: FILENAME = 'my_data.csv'

    ! 1. Read entire CSV into a 2D string array
    CALL read_csv_to_strings(FILENAME, .TRUE., ',', header_str, data_str,
        status)
```

```
    IF (status /= 0) STOP 'Error reading CSV'

    ! 2. Extract specific columns into typed arrays
    !    Let's say column 1 is Integer IDs, column 3 is Real scores
    CALL read_integer_columns(data_str, [1], data_int, status)
    IF (status /= 0) STOP 'Error reading integer columns'

    CALL read_real_columns(data_str, [3], data_real, status)
    IF (status /= 0) STOP 'Error reading real columns'

    ! The typed arrays data_int and data_real are now ready for analysis.
    PRINT *, 'Successfully processed data.'

END PROGRAM tox_fortran_example
```

3.2 Python Pipeline

The Python workflow is exposed through the `tensoromics_functions.py` module. The functions provide a high-level interface that mirrors the Fortran pipeline.

1. **Read CSV to Strings:** Call the `read_csv_as_strings` function. It takes the file path and returns the header and a 2D NumPy array of strings.
2. **Extract Typed Columns:** Pass the string NumPy array to the desired conversion functions, such as `read_integer_columns` or `read_real_columns`, along with a list of 1-based column indices to extract. All returned NumPy arrays are read-only for data safety.

Listing 2: Python Data Loading Workflow

```
import numpy as np
from tensoromics_functions import (
    read_csv_as_strings,
    read_integer_columns,
    read_real_columns
)

# Define the path to the data file
filename = 'my_data.csv'

try:
    # 1. Read the entire CSV into a NumPy array of strings
    header, data_str = read_csv_as_strings(filename)

    print("CSV data read successfully as strings:")
    print(data_str)

    # 2. Extract specific columns into new, typed NumPy arrays
    #    Column 1: Integer IDs
    #    Column 3: Real scores

    integer_data = read_integer_columns(data_str, columns=[1])
    real_data = read_real_columns(data_str, columns=[3])
```

```
    print("\nInteger data (column 1):")
    print(integer_data)

    print("\nReal data (column 3):")
    print(real_data)

    # The typed arrays are now ready for analysis (e.g., with SciPy,
       pandas)

except (FileNotFoundError, ValueError) as e:
    print(f"An error occurred: {e}")
```

3.3 R Pipeline

The R pipeline first reads the file into a raw, memory-efficient format (an integer array of ASCII codes) and then performs conversions from that raw format.

1. **Read CSV to Raw ASCII:** Call `tox.read_csv_raw`. This function reads the file and returns a list containing the data as a 3D integer array of ASCII codes. This intermediate format is efficient for passing between functions.
2. **Extract Typed Columns:** Pass the raw data array (`$data`) from the previous step to conversion functions like `tox.read_integer_columns` to get typed matrices.
3. **(Optional) Convert to Strings:** For inspection, you can convert the raw ASCII data to a human-readable character matrix using `tox.convert_ascii_to_strings`.

Listing 3: R Data Loading Workflow

```
# Load the TOX API functions
source("R/tensoromics_functions.R")

filename <- "my_data.csv"

# 1. Read the CSV file into a raw ASCII integer array format
raw_data_list <- tox.read_csv_raw(filename)

# For user inspection, convert the raw data to a character matrix
string_matrix <- tox.convert_ascii_to_strings(raw_data_list$data)
cat("Data as character matrix:\n")
print(string_matrix)

# 2. Use the raw_data_list$data to extract typed columns
#    Column 1: Integer IDs
#    Column 3: Real scores
integer_matrix <- tox.read_integer_columns(raw_data_list$data, columns =
   1)
real_matrix <- tox.read_real_columns(raw_data_list$data, columns = 3)

cat("\nExtracted integer data (column 1):\n")
print(integer_matrix)

cat("\nExtracted real data (column 3):\n")
```

```
print(real_matrix)
```